16

Trigger

16.1 Prozeduren automatisch ausführen

Ein **Trigger** ist eine besondere Form der Stored Procedures, bei der die Prozedur nicht durch den Anwender, sondern durch das Datenbanksystem selbst gestartet wird. Auf diese Weise können Sie einen Datenbestand überwachen und automatisch Aktionen ausführen, wenn Datensätze in Tabellen eingefügt, geändert oder gelöscht werden. Trigger bestehen aus drei wesentlichen Bestandteilen:

- Name des Triggers,
- Bedingung, die besagt, wann der Trigger aktiv wird,
- Ausführungsteil, ähnlich dem der Stored Procedures.

In der Bedingung eines Triggers kann festgelegt werden, ob die Anweisungen im Ausführungsteil bei einer `INSERT`-, einer `UPDATE`- oder einer `DELETE`-Anweisung aktiv werden. Der Ausführungsteil umfasst in der Regel mehrere SQL-Anweisungen, bei denen, wie bei Stored Procedures auch, Variablen und Kontrollstrukturen (`WHILE`, `IF` usw.) verwendet werden können.

Ein Trigger bezieht sich stets auf eine bestimmte Tabelle oder eine bestimmte Sicht. Es ist nicht möglich, Trigger zu definieren, die gleichzeitig Änderungen in verschiedenen Tabellen überwachen. Sie können jedoch für eine Tabelle mehrere Trigger erstellen, die verschiedene Ereignisse überwachen.

Nicht alle Integritätsregeln lassen sich deklarativ in der Datendefinition der Tabelle definieren. Mithilfe von Triggern ist es möglich, Integritätsregeln prozedural zu überprüfen und somit einen weiteren Beitrag zur Sicherung der referenziellen Integrität zu leisten. Beispielsweise kann das Einfügen ungültiger Datensätze bezüglich bestimmter Werte verhindert werden.

Zusätzlich zum SQL-Standard kann unter MariaDB und PostgreSQL festgelegt werden, ob der Trigger vor oder nach dem auslösenden Ereignis ausgeführt wird.

Schulversion
© HERDT-Verlag

16.2 Trigger erstellen

Trigger mit MariaDB erstellen

Trigger werden mit der Anweisung `CREATE TRIGGER` erstellt. Die Syntax ist dabei dem Definieren einer Stored Procedure ähnlich.

Beispiel MariaDB: *CreateTrigger.sql*

Mithilfe eines Triggers soll die Arbeit mit einem großen Datenbestand beschleunigt werden.

Die Anwender benötigen häufig die Summe der Stückzahlen und Preise aller Artikel des Lagerbestandes in der Tabelle *t_lager*. Die Werte berechnen Sie mit der Aggregatfunktion `SUMME`. Bei jedem Aufruf der Funktion werden alle Datensätze durchlaufen und die Werte zeitaufwendig und rechenintensiv addiert. Mit einem Trigger kann die Summe automatisch bei jedem Einfügen eines Datensatzes berechnet und in der zusätzlichen Tabelle *t_summe_ lager* gespeichert werden. Die Summe ist dort stets aktuell und schnell abrufbar.

```
① CREATE TABLE t_summe_lager
    (summe_stueck INTEGER,
     summe_preis FLOAT);
② DELIMITER //
③ CREATE TRIGGER trig_summe_artikel AFTER INSERT
④   ON t_lager FOR EACH ROW BEGIN
⑤     DECLARE summe_stueck INTEGER;
      DECLARE summe_preis FLOAT;
⑥      SELECT SUM(stueck), SUM(preis) FROM t_lager
         INTO summe_stueck, summe_preis;
⑦      DELETE FROM t_summe_lager;
⑧      INSERT INTO t_summe_lager
         VALUES(summe_stueck, summe_preis);
⑨ END //
⑩ DELIMITER ;
⑪ SELECT * FROM t_summe_lager;
   INSERT INTO t_lager (id, stueck, preis)
     VALUES(56,10, 200);
   SELECT * FROM t_summe_lager;
```

① Zuerst wird eine neue Tabelle *t_summe_lager* erstellt, die für jede der benötigten Summen ein Datenfeld erhält.

② Wie bei Stored Procedures muss auch bei der Definition eines Triggers durch `DELIMITER` ein Endekennzeichen benannt werden, da sonst die Eingabe des Triggers im Datenbank-Client nicht möglich ist.

③ Hier beginnt die Definition des Triggers *trig_summe_artikel* mit der Anweisung `CREATE TRIGGER`. Die Angabe der Schlüsselwörter `AFTER INSERT` bewirkt, dass der Trigger stets nach dem Einfügen eines neuen Datensatzes aktiv wird.

© HERDT-Verlag
Schulversion

④ Der Trigger soll die Tabelle *t_lager* überwachen. Dies wird mit dem Schlüsselwort ON erreicht. Ab dem Schlüsselwort BEGIN erfolgt die Angabe des Anweisungsteils. Dieser entspricht dem einer Stored Procedure.

⑤ Zuerst werden dazu zwei lokale Variablen deklariert.

⑥ Die Summe der Datenfelder wird mit einer Anweisung SELECT INTO ermittelt und in den Variablen gespeichert.

⑦ Danach wird der möglicherweise vorhandene alte Datensatz der Tabelle *t_summe_lager* gelöscht.

⑧ Die neu berechneten Summen werden mittels der Variablen in die Tabelle *t_summe_lager* eingefügt.

⑨ Die Deklaration der Prozedur endet mit END. Die Anweisung muss entsprechend der Neudefinition des Endekennzeichens (②) mit dem Zeichen // abgeschlossen werden.

⑩ Nun kann als Endekennzeichen wieder das Semikolon definiert werden.

⑪ Zur Überprüfung wird der Inhalt der Tabelle *t_summe_ lager* abgefragt, danach wird ein neuer Artikel eingefügt und damit die Arbeit des Triggers überprüft.

```
MariaDB [uebungen]> SELECT * FROM t_summe_lager;
+--------------+-------------+
| summe_stueck | summe_preis |
+--------------+-------------+
|         1558 |     374.073 |
+--------------+-------------+
1 row in set (0.00 sec)

MariaDB [uebungen]> INSERT INTO t_lager (id, stueck, preis)
    ->     VALUES(57, 15, 220);
Query OK, 1 row affected (0.01 sec)

MariaDB [uebungen]> SELECT * FROM t_summe_lager;
+--------------+-------------+
| summe_stueck | summe_preis |
+--------------+-------------+
|         1573 |     594.073 |
+--------------+-------------+
1 row in set (0.00 sec)
```

Der Trigger hat die neuen Summen berechnet und eingefügt

Syntax

```
CREATE TRIGGER triggername { BEFORE | AFTER } {INSERT|UPDATE|DELETE}
    ON tabellenname FOR EACH ROW BEGIN
       anweisungsblock
        END;
```

- Die Anweisung beginnt mit den Schlüsselwörtern CREATE TRIGGER. Danach folgt der Name des Triggers.
- Eines der Schlüsselwörter BEFORE oder AFTER wird verwendet, um festzulegen, ob der Trigger vor (BEFORE) oder nach (AFTER) der angegebenen Aktion aktiv werden soll.
- Mit einem der Schlüsselwörter INSERT, UPDATE oder DELETE wird die Art der Datenmodifikation benannt, bei der der Anweisungsteil des Triggers ausgeführt werden soll.
- Mit dem Schlüsselwort ON wird der Name der Tabelle bzw. der Sicht angegeben, für die der Trigger gelten soll.
- Die Schlüsselwörter FOR EACH ROW bedeuten, dass der Trigger nicht nur ein einziges Mal auf die gesamte Tabelle angewendet wird, sondern jeden einzelnen Datensatz anspricht, der von der Anweisung betroffen ist.
- Nach dem Schlüsselwort BEGIN werden, falls benötigt, lokale Variablen deklariert.
- Zwischen BEGIN und END steht der eigentliche Anweisungsteil des Triggers.
- Bei der Eingabe im Datenbank-Client muss die Anweisung mit einem Endekennzeichen, standardmäßig dem Semikolon, beendet werden.

© HERDT-Verlag

Bei SQL dürfen keine zwei Trigger einer Tabelle dieselbe Aktionszeit und dasselbe Trigger-Ereignis haben. Zum Beispiel können Sie keine zwei BEFORE UPDATE-Trigger für eine Tabelle definieren. Sie können jedoch einen BEFORE UPDATE- und einen BEFORE INSERT-Trigger oder einen BEFORE UPDATE- und einen AFTER UPDATE-Trigger definieren.

Trigger mit PostgreSQL erstellen

Auch PostgreSQL unterstützt Trigger. Dabei ist jedoch zu beachten, dass hier kein einfacher Anweisungsblock übergeben werden kann, sondern eine benutzerdefinierte Funktion mit dem Rückgabewert *trigger* angegeben werden muss, die bereits zuvor programmiert wurde.

Seit der Version 9.3 unterstützt PostgreSQL sogenannte Event Trigger, die in Abhängigkeit von DDL-Befehlen (CREATE, ALTER, DROP) ausgeführt werden. Diese Funktionalität ist eine PostgreSQL-Besonderheit, welche nicht auf dem SQL-Standard basiert.

Syntax

```
CREATE TRIGGER triggername { BEFORE | AFTER } {INSERT|UPDATE|DELETE}
    ON tabellenname [ OF spaltenname ] FOR EACH ROW
    WHEN bedingung
     EXECUTE PROCEDURE funktion ( [ argument [, ...] ] )
```

- Die Anweisung beginnt mit den Schlüsselwörtern CREATE TRIGGER. Danach folgt der Name des Triggers.
- Eines der Schlüsselwörter BEFORE oder AFTER wird verwendet, um festzulegen, ob der Trigger vor (BEFORE) oder nach (AFTER) der angegebenen Aktion aktiv werden soll.
- Mit einem der Schlüsselwörter INSERT, UPDATE oder DELETE wird die Art der Datenmodifikation benannt, bei der der Anweisungsteil des Triggers ausgeführt werden soll.
- Mit dem Schlüsselwort ON wird der Name der Tabelle bzw. der Sicht angegeben, für die der Trigger gelten soll.
- Optional kann eine Spalte angegeben werden. Ist dies der Fall, wird der Trigger nur ausgeführt, wenn der Wert in der betreffenden Spalte aktualisiert wird.
- Die Schlüsselwörter FOR EACH ROW bedeuten, dass der Trigger nicht nur ein einziges Mal auf die gesamte Tabelle angewendet wird, sondern jeden einzelnen Datensatz anspricht, der von der Anweisung betroffen ist.
- Es kann eine zusätzliche Bedingung angegeben werden, um die Ausführungshäufigkeit des Triggers weiter einzuschränken. Nur wenn die Bedingung erfüllt ist, wird der Trigger ausgeführt.
- Nach dem Schlüsselwort EXECUTE PROCEDURE und dem betreffenden Funktionsnamen wird eine Funktion aufgerufen.
- Die Argumente, die der Funktion übergeben werden sollen, werden anschließend in Klammern angegeben. Dabei hängt die Art und Weise, wie die Triggerargumente in der Funktion ermittelt werden können, von der jeweiligen Implementierungssprache der Funktion ab.

Informationen zur Programmierung von Funktionen mit PostgreSQL finden Sie in dem betreffenden Handbuch.

Bisherige und neue Datenfeldinhalte in Triggern auslesen

Beim Einfügen neuer Werte oder dem Aktualisieren bestehender Werte im Anweisungsteil eines Triggers kann es notwendig sein, auf den neuen oder den alten Inhalt eines bestimmten Datenfelds zuzugreifen. Dafür können Sie unter MariaDB und PostgreSQL die Transitionsvariablen NEW bzw. OLD in Kombination mit einem Datenfeldnamen verwenden.

NEW.datenfeldname	Liefert den neuen oder zu ändernden Wert eines Datenfeldes in einem Trigger für die INSERT- oder UPDATE-Aktion
OLD.datenfeldname	Liefert den bisherigen Wert eines Datenfeldes in einem Trigger für die UPDATE- und DELETE-Aktion

Beispiel für MariaDB: *CreateTriggerNew.sql*

Beim Einfügen von Datensätzen in die Tabelle *t_liefer* muss eine eindeutige Zahl als Lieferantennummer angegeben werden. Mithilfe eines Triggers soll erreicht werden, dass eine neue Nummer erzeugt wird, falls bei einer INSERT-Anweisung keine Angabe erfolgt.

```
   DELIMITER //
①  CREATE TRIGGER trig_liefer_nr BEFORE INSERT
     ON t_liefer FOR EACH ROW BEGIN
       DECLARE lid INTEGER;
②      IF (NEW.id = 0) THEN
③        SELECT MAX(id)+1 FROM t_liefer INTO lid;
④        SET NEW.id = lid;
       END IF;
     END //
   DELIMITER ;
⑤  INSERT  INTO t_liefer (name, str, hnr, plz, ort)
         VALUES ("Walter", "Hauptstr.", "6", "60320", "Frankfurt");
   SELECT * FROM t_liefer;
```

① Der Trigger wird mit der CREATE TRIGGER-Anweisung für die Tabelle *t_liefer* erstellt. Da das Feld *id* nicht den Wert NULL enthalten soll, muss die Überprüfung durch den Trigger noch vor dem tatsächlichen Einfügen erfolgen. Dies wird durch die Angabe der Schlüsselwörter BEFORE INSERT erreicht.

② In einer IF THEN-Anweisung erfolgt mithilfe der Transitionsvariable NEW die Prüfung, ob für das Feld *id* eine Eingabe erfolgt ist. Ist das nicht der Fall, wird der folgende Anweisungsblock ausgeführt.

③ In einer SELECT-Anweisung wird der maximale Wert für das Feld *id* ermittelt, um den Wert 1 erhöht und in der Variablen *lid* gespeichert.

④ Über die Transitionsvariable NEW wird dem Feld *id* der Wert der Variablen *lid* zugewiesen.

⑤ Um den Trigger zu testen, wird ein Datensatz ohne Eintrag für die *id* eingegeben. Anschließend wird geprüft, ob die fehlende *id* ergänzt wurde.

© HERDT-Verlag

16.3 Trigger bearbeiten und löschen

Trigger ändern

Mit der Anweisung `ALTER TRIGGER` können Sie in einigen Datenbanksystemen einen bestehenden Trigger bearbeiten. Dabei ist es möglich, einen neuen Anweisungsteil anzugeben, die Positionsnummer oder die auslösende Aktion zu ändern. Sie können Kopf- und Anweisungsteil des Triggers gleichzeitig oder einzeln bearbeiten.

- Mit der `ALTER TRIGGER`-Anweisung können Sie nicht die Tabelle tauschen, an die der Trigger gebunden ist. Dies ist nur möglich, indem Sie den Trigger löschen und vollständig neu erstellen.
- Um einen Trigger zu bearbeiten, müssen Sie der Besitzer des Triggers sein, ihn erstellt haben oder Datenbankadministrator sein.
- MariaDB unterstützt diese Anweisung nicht. Wenn Sie einen Trigger in MariaDB ändern wollen, müssen Sie ihn löschen und neu erstellen.

Syntax MariaDB

```
ALTER TRIGGER triggername [{BEFORE|AFTER} {INSERT|UPDATE|DELETE}]
```

Die anschließende Syntax richtet sich nach dem jeweiligen Datenbanksystem.

- Die Anweisung beginnt mit den Schlüsselwörtern `ALTER TRIGGER`, gefolgt vom Namen des zu bearbeitenden Triggers.
- Optional kann danach die auslösende Aktion des Triggers neu angegeben werden. Dafür wird eines der Schlüsselwörter `BEFORE` oder `AFTER` verwendet, um den Zeitpunkt festzulegen, und eine der Angaben `INSERT`, `UPDATE` oder `DELETE` für die Art der zu überwachenden Datenmodifikation.
- Nach dem Schlüsselwort `BEGIN` folgt optional ein neuer Anweisungsblock.

Verwenden Sie zur Definition eines Triggers einen separaten Editor, besitzen Sie mehr Möglichkeiten zur Texteingabe und Änderung. Nach der Fertigstellung der Triggerdefinition kopieren Sie diese in das entsprechende Programm, um den Trigger zu erstellen. Für geringe Änderungen ist die Anweisung `ALTER TRIGGER` geeignet.

Bei PostgreSQL können Sie zusätzlich zu den beschriebenen Änderungen mit der Anweisung `RENAME TO` auch den Namen des Triggers ändern.

Syntax PostgreSQL

```
ALTER TRIGGER triggername ON tabelle RENAME TO neuername
```

Trigger löschen

Falls ein Trigger nicht mehr benötigt wird oder Sie ihn vollständig neu erstellen wollen, können Sie ihn mit der DROP TRIGGER-Anweisung aus dem System löschen.

Um einen Trigger zu löschen, müssen Sie der Besitzer des Triggers sein, ihn erstellt haben oder der Datenbankadministrator sein.

Syntax MariaDB

```
DROP TRIGGER [tabellenname] triggername ;
```

- Die Anweisung beginnt mit den Schlüsselwörtern DROP TRIGGER. Optional kann der Name der Tabelle genannt werden, für die der Trigger erstellt wurde. Danach folgt der Name des zu löschenden Triggers.

Syntax PostgreSQL

```
DROP TRIGGER triggername ON tabellenname [ CASCADE | RESTRICT ];
```

- Die Anweisung beginnt mit den Schlüsselwörtern DROP TRIGGER, gefolgt von dem Namen des Triggers.
- Bei PostgreSQL ist die Angabe des Tabellennamens, für die der Trigger erstellt wurde, verpflichtend. Er wird eingeleitet durch das Schlüsselwort ON.
- Mit der Angabe CASCADE werden alle Objekte, die von dem betreffenden Trigger abhängen, gelöscht. Wenn Sie verhindern möchten, dass der Trigger gelöscht wird, solange noch Objekte von dem Trigger abhängen, geben Sie die Option RESTRICT an.

16.4 Übung

Trigger erstellen

Übungsdatei: -- **Ergebnisdatei: *Uebung.sql***

1. Erstellen Sie in der Datenbank *Uebungen* einen neuen Trigger *trig_summe_artikel_update*.
2. Der Trigger soll die Summe der Stückzahlen und Preise aller Artikel am Lager berechnen, wenn ein Daten-satz in der Tabelle *t_lager* aktualisiert wird.
3. Berechnen Sie die Summen mithilfe einer Aggregatfunktion.
4. Speichern Sie die Werte in der Tabelle *t_summe_lager*. Löschen Sie gegebenenfalls alte Einträge in der Tabelle.
5. Prüfen Sie die Funktionsfähigkeit, indem Sie einige Stückzahlen und Preise in der Tabelle verändern.
6. Berechnen Sie die Summen mit einer SELECT-Abfrage und vergleichen Sie das Ergebnis mit den Werten in der Tabelle *t_summe_lager*.
7. Erstellen Sie den gleichen Trigger für die DELETE-Aktion.

Schulversion
© HERDT-Verlag